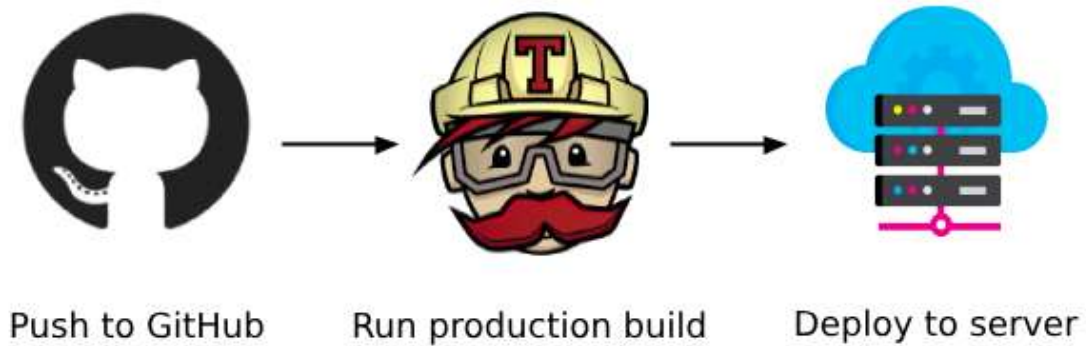


Lab 04 - Configuring Travis



In last week's exercise we signed up for a Travis account using GitHub for the account creation process. We also activated our comp3104 repository with Travis by toggling the green button. If you have not yet done this, please take care of it in this lab class. If you require any assistance, please let me know.

If you've been following along, you should see an output like mine in your Travis CI account:

jayoharedee / comp3104 build failing

[Current](#) [Branches](#) [Build History](#) [Pull Requests](#) More options

✖ master added travis.yml

Commit 3c7f3b7 [↗](#)

Compare 319ffe7...3c7f3b7 [↗](#)

Branch master [↗](#)

jayoh

-o- #1 failed

Ran for 14 sec

3 minutes ago

Restart build

`</>` Ruby

This is an indication that we have a failing build in case the UI elements have not made the status apparent. The reason for our failed build is due to us asking Travis to watch our repo and it finding an empty configuration file. Let's talk a little bit about what a Travis config file is, how it works and then work on getting our build to pass.

Working with Travis

In order for us to work with Travis, two steps must be taken:

1. Telling Travis to watch our repository
2. Adding a `.travis.yml` file to our repository

When Travis CI has been activated for a given repository, GitHub will notify Travis whenever new commits are pushed to the master/main branch, or as best practice dictates, when a PR is approved, and code is merged into the master/main branch.

Travis will be acting as our Continuous Integration suite and perform any steps we instruct it to through the configuration file in our repository - `.travis.yml`. It can complete anything from running tests, building the application or running deployment scripts.

We know that Continuous Integration is the act of committing changes to the source code and with each commit made, we trigger a build of our application. This practice allows teams to detect problems early and also include other sequences to take place like a static code analysis or the execution of a test suite.

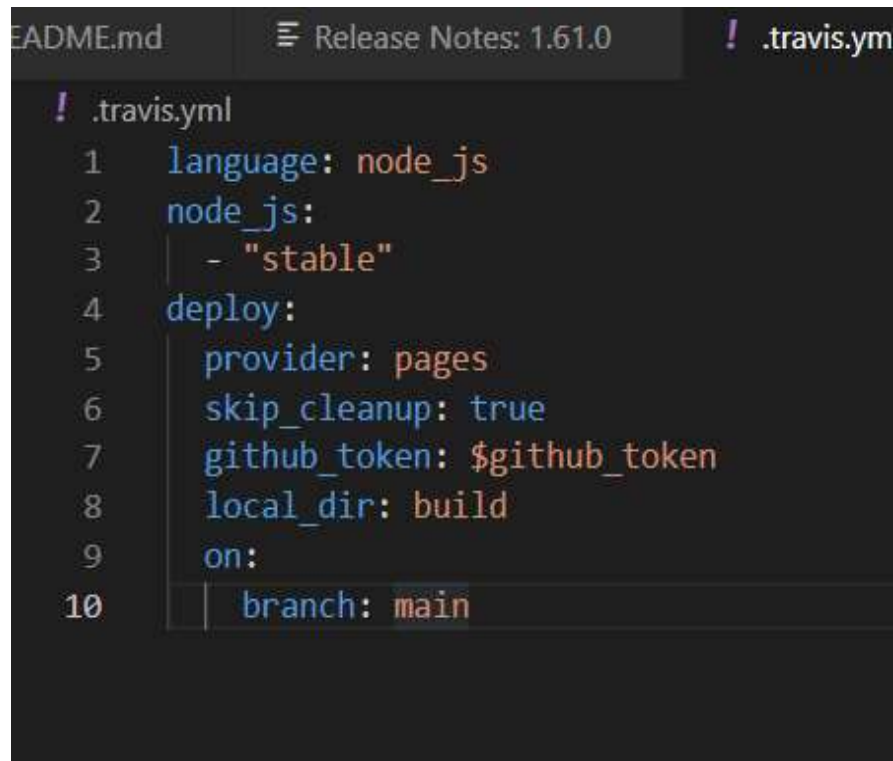
Let's test this theory out by adding a change to our Travis config file and pushing the change to our master/main branch. **Please use the minimal configuration found below and add the following to your `.travis.yml` file in your comp3104 project directory.**

,

```
language: node_js
node_js:
  - "stable"
deploy:
  provider: pages
  skip_cleanup: true
  github_token: $github_token
  local_dir: build
on:
  branch: main
```

,

The above text is only intended for ease of access to copy and paste but please be aware of any spacing as we are dealing with YAML format. YAML stands for “Yet Another Markup Language” for more information on this standard, please reference the labs supplementary material section. One thing to know is that it relies on block scoping similar to Python and relies on nesting of nodes so please pay attention to the spacing found in the image below.



```
! .travis.yml
1  language: node_js
2  node_js:
3    - "stable"
4  deploy:
5    provider: pages
6    skip_cleanup: true
7    github_token: $github_token
8    local_dir: build
9    on:
10   | branch: main
```

After committing the configuration change and pushing to our master/main branch, we should run into a couple more failures, but let's work through them together.

If we take a look at the job that was triggered after pushing our recent change in our `.travis.yml` config file, we should see the following failure:

```
1 Worker information
6 Build system information
158
159
160 $ git clone --depth=50 --branch=master https://github.com/jayoharedee/comp3104.git
170
171 $ nvm install stable
177
178 $ node --version
179 v12.6.0
180 $ npm --version
181 6.9.0
182 $ nvm --version
183 0.34.0
184
185 $ make test
186 make: *** No rule to make target 'test'. Stop.
187 The command "make test" exited with 2.
188
189
190
191 Done. Your build exited with 1.
```

he exact error thrown was due to Travis expecting a test case, but we did not provide one, thus the job failed when no test was found. Let's use npm to initialize our comp3104 project with Node. Doing so will create a package.json file in our project directory which will allow us to write both test and build scripts.

Head over to your terminal and please ensure you are in your project directory. Once you're in the comp3104 project, you will want to enter the following command:

```
`npm init -y`
```

The resulting output should produce something like what's found in my terminal after executing the above command.

```
ghost@Garrys-MacBook-Pro ~/comp3104 master npm init -y
Wrote to /Users/ghost/comp3104/package.json:

{
  "name": "comp3104",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/jayoharedee/comp3104.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/jayoharedee/comp3104/issues"
  },
  "homepage": "https://github.com/jayoharedee/comp3104#readme"
}
```

It's just a little bit of JSON, something you may have seen before consisting of key/value pairs. Please take a look at the 5th key entitled "scripts". Notice the object assignment containing a key called "test". We're going to want to open the newly generated package.json and replace the value for "test" with the following:

```
`"echo \"Warning: No tests created yet.\"`
```

I'm using VS Code for an editor now, for yourself, please use whatever your most comfortable with to edit the file. After replacing the value for the "test" key, my JSON now looks like this:

```
package.json x
comp3104 > package.json > {} scripts > abc test
1  {
2    "name": "comp3104",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Warning: No tests created yet.\""
8    },
9    "repository": {
10     "type": "git",
11     "url": "git+https://github.com/jayoharedee/comp3104.git"
12   },
13   "keywords": [],
14   "author": "",
15   "license": "ISC",
16   "bugs": {
17     "url": "https://github.com/jayoharedee/comp3104/issues"
18   },
19   "homepage": "https://github.com/jayoharedee/comp3104#readme"
20 }
```

Please note line 7, I am simply echoing out the fact that we have not yet created any test cases. This should now echo out in our Travis log when we commit this new change to our master/main branch in the comp3104 repo. Let's commit this change and push it up to our master/main branch.

✕ Remove log ⌵ Raw log

▶ 1 Worker information worker_info

▶ 6 Build system information system_info

158

docker_mtu

resolvconf

159

▶ 160 \$ git clone --depth=50 --branch=master https://github.com/jayoharedee/comp3104.git git.checkout 0.74s

170

▶ 171 \$ nvm install stable nvm.install 4.39s

177

178 \$ node --version

179 v12.6.0

180 \$ npm --version

181 6.9.0

182 \$ nvm --version

183 0.34.0

184

▶ 185 \$ npm install install.npm 0.94s

192

193 \$ npm test 0.23s

194

195 > comp3104@1.0.0 test /home/travis/build/jayoharedee/comp3104

196 > echo "Warning: No tests created yet."

197

198 Warning: No tests created yet.

199 The command "npm test" exited with 0.

200

201

▶ 202 \$ rvm \$(travis_internal_ruby) --fuzzy do ruby -S gem install dpl dpl_0 1.60s

206 invalid option "--github_token="

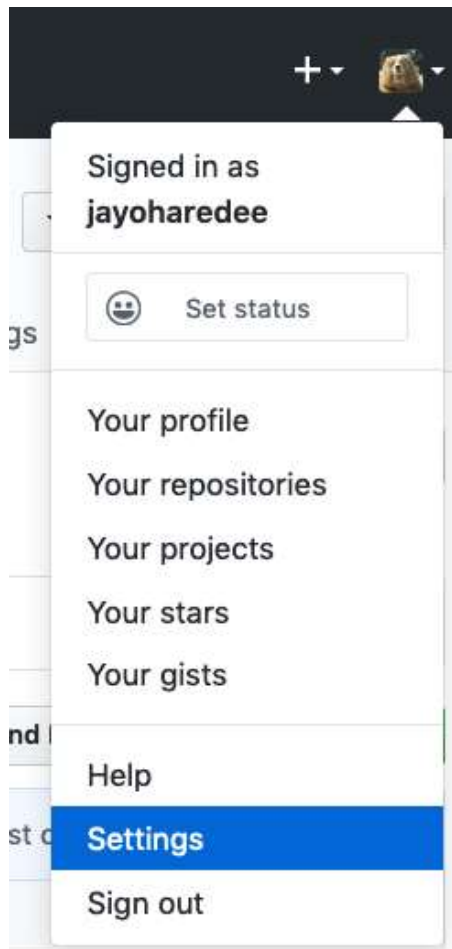
207 failed to deploy

Top ▲

Alright! Seeing some green with an exit of 0 is good but we still have one last error to deal with. Notice how it mentions something about an invalid option for `--github_token=` ? Let's take care of this by going back to our travis config file for a moment.

```
You, an hour ago | 1 author (You)
1  language: node_js
2  node_js: |                                You, 2 hours ago
3      - "stable"
4  deploy:
5      provider: pages
6      skip_cleanup: true
7      github_token: $github_token
8      local_dir: build
9      on:
10         branch: master
```

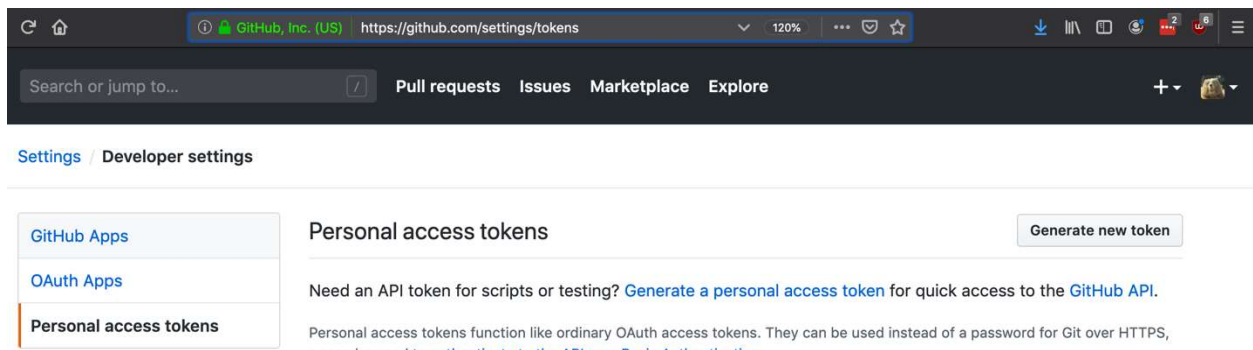
Looking at line 7, we can see a node called `github\_token:` with a value of `\$github\_token`. This value is intended to be an environment variable containing a personal access token which GitHub will provide us for the comp3104 repository. This token grants Travis permission to interact with our repository. Head to the same settings area we used when assigning our public SSH key.



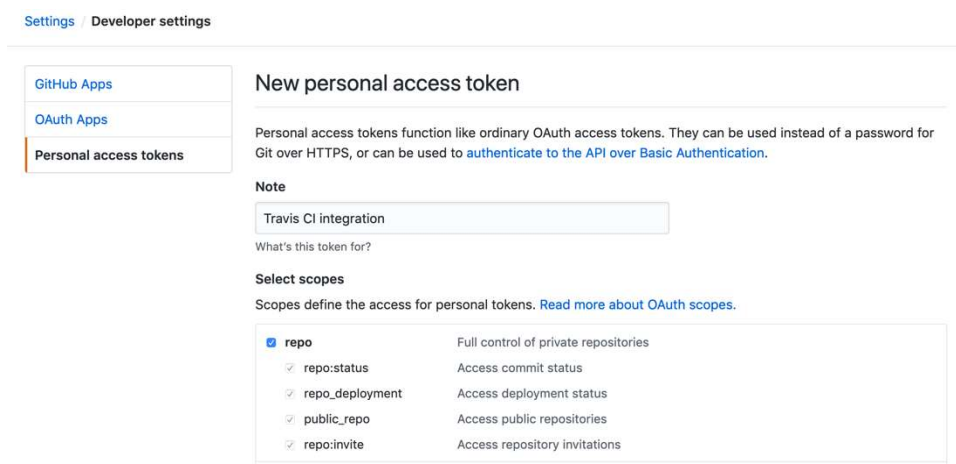
After accessing the settings area, you should see an item in the side menu that says 'Developer settings. We need to access the Developer Settings followed by another side menu navigation click on 'Personal access tokens'. If you're logged in, you should be able to access this page with the following URL:

<https://github.com/settings/tokens>

Once on the personal access tokens page, we're going to need to generate a new token by clicking the grey button titled "Generate new token". See the below image for more details.



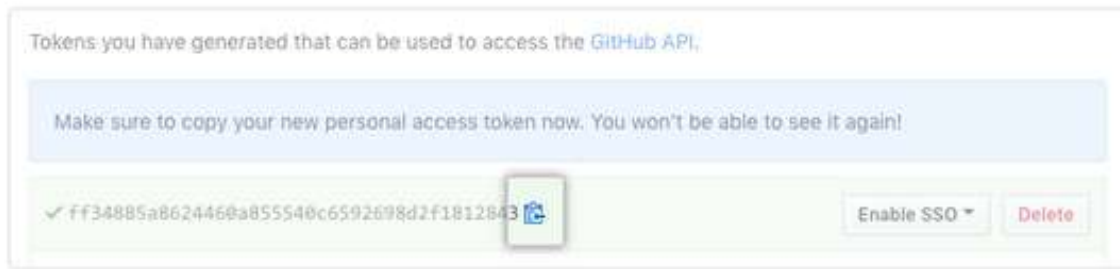
You'll then want to select the repo scope to grant permissions for use.



Now it's time to generate the token by clicking the big green button below



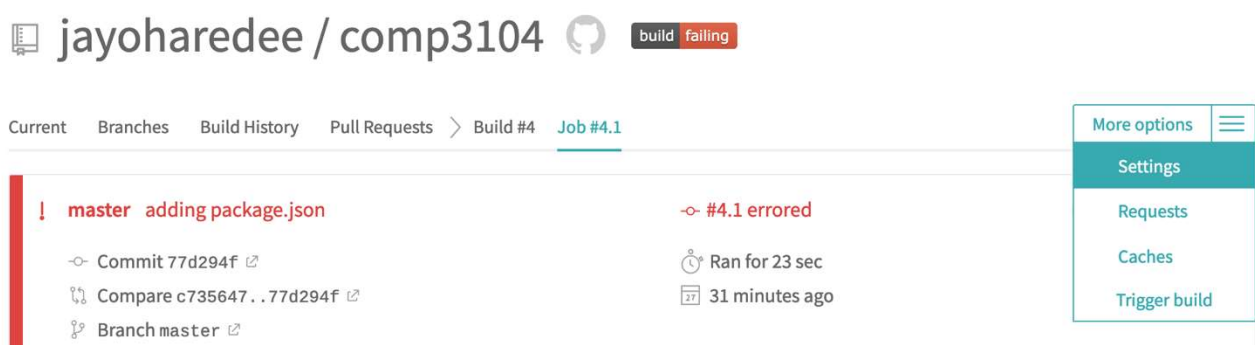
After generating the token, we'll want to keep this private so don't share your token with others. Click the clipboard as demonstrated in the below image.



Warning: Treat your tokens like passwords and keep them secret. When working with the API, use tokens as environment variables instead of hardcoding them into your programs.

Once we have the copied token, we'll want to head back over to our Travis console and find our settings page to create the `$github_token` environment variable. I've gone ahead and documented the process for this so please follow the images below.

First step; head back to the Travis CI console where our build log was produced and look in the top right corner for a menu that says 'More Options' and click on the 'Settings' link in the drop down menu.



After being presented with the Settings page, if you scroll down about half way you should be presented with a section entitled "Environment Variables". Here we'll be entering our key which is `github_token` and our value is the copied token from GitHub taken from our previous step.

Environment Variables

Customize your build using environment variables. For secure tips on generating private keys [read our documentation](#)

📢 If your secret variable has special characters like `&`, escape them by adding `\` in front of each special character. For example, `ma&w!doc` would be entered as `ma\&w\!doc`.

NAME	VALUE		DISPLAY VALUE IN BUILD LOG	
Name	Value	☐		<button>Add</button>

Here's what mine looks like after filling it out. Ensure that the toggle is not active just like mine is below:

Environment Variables

Customize your build using environment variables. For secure tips on generating private keys [read our documentation](#)

📢 If your secret variable has special characters like `&`, escape them by adding `\` in front of each special character. For example, `ma&w!doc` would be entered as `ma\&w\!doc`.

NAME	VALUE		DISPLAY VALUE IN BUILD LOG	
github_token	234ghj324hgj34hcg23g23jkh55jkh534jkh34j4k3h	☐		<button>Add</button>

We should now have a newly created environment variable that we can use in our `.travis.yml` config file. After clicking the “Add” button, you should be presented with something like this:

Environment Variables

Customize your build using environment variables. For secure tips on generating private keys [read our documentation](#)

github_token		
--------------	-------	--

Now, to test the above out, we're going to create a new folder in our project directory called build. To do this, we'll be on the command line and in our project directory using the `mkdir` command we used in our first lab.

Let's hop over to our terminal and enter the following commands to create a folder along with an index.html file.

```
mkdir -p build
```

``cd build``

``touch index.html``

```
ghost@Garrys-MacBook-Pro ~/comp3104 master mkdir -p build ✓ 388 18:06:37
ghost@Garrys-MacBook-Pro ~/comp3104 master cd build ✓ 389 18:06:43
ghost@Garrys-MacBook-Pro ~/comp3104/build master touch index.html ✓
ghost@Garrys-MacBook-Pro ~/comp3104/build master ? ✓ 391 18:06:58
```

Let's now push this newly created directory and file upstream and see what happens.

If everything went as planned, here is the process we should have seen take place:

First, our CI suite boots up a container to run our process in:

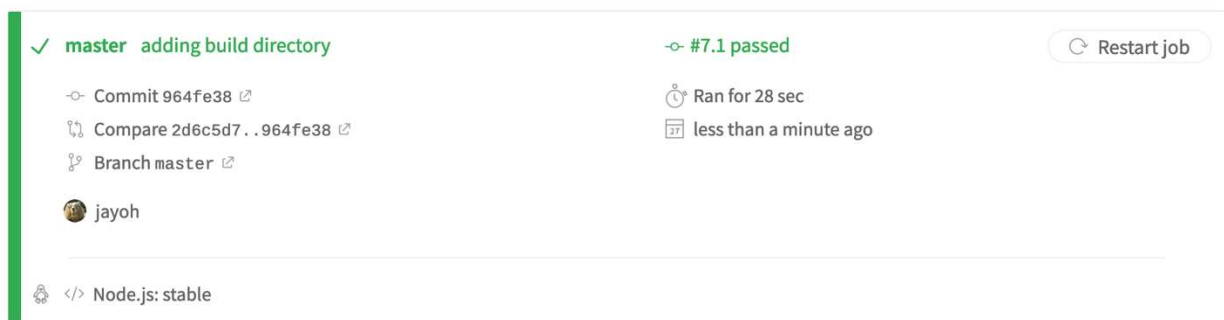
The screenshot shows a CI/CD interface for a job named "master adding build directory". The job is currently in the "Waiting to start" state, indicated by a clock icon. The interface includes a "Cancel job" button and a "View config" link. Below the job details, there is a progress bar with three steps: 1. Job received, 2. Queued, and 3. Booting virtual machine. The progress bar shows that the job has moved through the first two steps and is currently in the third step.

After this completes, we should start seeing some output in our log, the end of which looks like this:

```
191 $ npm test
192
193 > comp3104@1.0.0 test /home/travis/build/jayoharedee/comp3104
194 > echo "Warning: No tests created yet."
195
196 Warning: No tests created yet.
197 The command "npm test" exited with 0.
198
199
▶ 200 $ rvm $(travis_internal_ruby) --fuzzy do ruby -S gem install dpl dp1_0 1.63s
203 5.25s
▶ 204 Installing deploy dependencies dp1.1
214 Logged in as @jayoharedee (jayoh)
215 cd /tmp/d20190704-5191-1ljcupn/work
▶ 216 Preparing deploy dp1.2
▶ 218 Deploying application dp1.3
223 Done. Your build exited with 0.
Top ▲
```

Alright, so judging by the output above it looks like our application was successfully deployed as there is no errors. Please note on line 214 above that Travis actually performs a login process with the token we provided earlier. We can see my username displayed that's associated with the token.

At the beginning of this lab, we've seen what a failed build looks like. Hopefully now, your build is passing too:



To ensure that everything is truly copacetic, we're going to perform one last step. We're going to add a badge to our README.md file to indicate the status of our latest build.



Looking at the image above, we can see a badge that says, 'build error'. Let's click on this badge in order to get the following pop-up:

Status Image

BRANCH

master

MARKDOWN

`[![Build Status](https://travis-ci.com/jayoharedee/comp3104.svg?branch=master)](https://travis-ci.com/jayoharedee/comp3104)`

You'll want to select the second select element and choose the MARKDOWN option just as I've done above. Take the value from the text field and copy that bit of markdown. We'll then want to go and paste it into our README.md file as shown below.

```
comp3104 > README.md > abc #### COMP3104 - DevOps
You, a few seconds ago | 1 author (You)
1  #### COMP3104 - DevOps
2
3  [![Build Status](https://travis-ci.com/jayoharedee/comp3104.svg?branch=master)](https://travis-ci.com/jayoharedee/comp3104)
```

Let's push this upstream and see what change has been made to our projects readme.

The resulting push should now display the following in our projects readme file:

README.md

COMP3104 - DevOps

build passing

Creating a pull request template for your repository

- 1) On you main branch add `PULL_REQUEST_TEMPLATE.md` in `.github` folder
- 2) Add below content to the file

Summary

Briefly describe the change and why it's needed.

Related Issue / Ticket

Closes #<issue-number> (or link)

Type of change

- [] Feature

- [] Bug fix

- [] Refactor / Chore

- [] Docs

- [] Test

How was this tested?

Describe test strategy, coverage, and any manual verification steps.

Screenshots / Demos (if applicable)

<insert images, GIFs, or links>

Checklist

- [] I ran the test suite locally and it passed.

- [] I updated docs (README/CHANGELOG) if needed.

- [] I followed project coding standards/linting.

- [] I've assigned reviewers and set labels/milestones as appropriate.

3) Now commit the change to main branch

Adding GitHub action status badge to your repo (NOT needed for Travis CI)

- 1) Copy below line to your README.md file and update username with your GitHub username. *If needed also change repo name and action file name.*
- 2) commit/push to your remote branch

[![CI]

(https://github.com/ORG_OR_USERNAME/COMP3104/actions/workflows/ci.yml/badge.svg)]

(https://github.com/ORG_OR_USERNAME/COMP3104/actions/workflows/ci.yml)

Will see how the pull request template works next week when we will create new branch